Programowanie współbieżne w C++

Lista zadań nr 5

Na pracownię 2 kwietnia 2026

Uwaga: W zadaniu 1 należy stosować blokujące środki synchronizacji (muteksy). W pozostałych zadaniach stosować ich nie wolno (należy wykorzystać `std::atomic`).

Zadanie 1 (5pkt) Współbieżny graf relacji (rozszerzenie).

Rozszerz swoją implementację współbieżnego grafu relacji (zadanie 2 z listy 4) o następujące funkcje:

1. `bool is_path(const std::vector<int> &p) const;` sprawdzającą czy wierzchołki z p tworzą ścieżkę w grafie
2. `bool is_path_unsafe(int id_a, int id_b) const;` sprawdzającą, czy w grafie istnieje ścieżka pomiędzy wierzchołkami `id_a` a `id_b`. Ta implementacja dopuszcza zakleszczenie pomiędzy wątkiem wywołującym tą funkcję a wątkami modyfikującymi graf. Funkcja powinna blokować odwiedzone wierzchołki w trybie `shared`.
3. `bool is_path_safe(int id_a, int id_b) const;` również sprawdzającą istnienie ścieżki, ale w wersji bez zakleszczeń. Zastosuj tu podejście optymistyczne. Znajdź najpierw kandydata na ścieżkę, a następnie zweryfikuj, że faktycznie jest to ścieżka. Zastanów się, w jakiej sytuacji funkcja powinna zwrócić false, a w jakiej wykonać się ponownie.

Wszystkie funkcje powinny stosować blokowanie drobnoziarniste (na poziomie wierzchołka) oraz przechowywać niezbędne dane pomocnicze lokalnie, a nie w strukturze wierzchołka.

Zadanie 2 (4pkt). Kolejka dwuwątkowa.

Zaimplementuj kolejkę dwuwątkową (jednocześnie co najwyżej jeden wątek może wykonywać operację `push` oraz co najwyżej jeden może wykonywać `pop`) w wersji wait-free. Użyj bufora cyklicznego (tablicy) o stałym rozmiarze oraz operacji atomowych. Przetestuj swoją implementację, a w szczególności warunki brzegowe.

Zadanie 3 (4pkt). Pula elementów.

Zaimplementuj pulę elementów z operacjami `put` i `get` używając operacji atomowych, tablicy o stałym rozmiarze i dwóch liczników. Przetestuj swoją implementację, a w szczególności warunki brzegowe.

Wskazówka: The Art of Multiprocessor Programming 2e, r. 12.2.

Zadanie 4 (5pkt). Współbieżny stos oparty na inteligentnych wskaźnikach (C++20).

Zaimplementuj wielowątkowy stos, oparty na liście jednokierunkowej elementów typu `Node`, łączonej dynamicznie. Zamiast surowych wskaźników, do łączenia węzłów i wskazywania szczytu stosu użyj wprowadzonych w standardzie C++20 atomowych wskaźników współdzielonych: `std::atomic<std::shared_ptr<Node>>`. Zaimplementuj operacje `push(T data)` oraz `std::shared_ptr<T> pop()`, realizując je przy pomocy bezpiecznych pętli i metody `compare_exchange_weak`. W komentarzu do swojego kodu udziel odpowiedzi na pytania:

1. Dlaczego użyto wskaźnika współdzielonego (`shared_ptr`), a nie unikalnego (`unique_ptr`), który wydawałby się lżejszy. Dlaczego kompilator nie pozwoliłby Ci skompilować kodu ze zmienną `std::atomic<std::unique_ptr<Node>>`?
2. Pokaż za pomocą metody `is_lock_free()`, czy Twoja implementacja z użyciem `std::atomic<std::shared_ptr<Node>>` faktycznie okazała się bezblokadowa (lock-free) w systemie, którego używasz. Co oznacza ewentualny wynik `false` w kontekście wydajności Twojej struktury?
3. Dlaczego nie chcemy tutaj użyć `std::atomic<Node*>`?

Przetestuj swoją implementację, a w szczególności warunki brzegowe.

Wskazówka: The Art of Multiprocessor Programming 2e, r. 11.2.

Zadanie 5 (5pkt). Klasa LockFreeExchanger.

Zaimplementuj klasę LockFreeExchanger, służącą do wymiany danych pomiędzy parami wątków z ograniczeniem czasowym. Obiekty tej klasy powinny być wielokrotnego użytku, tzn. po zakończeniu wymiany obiekt powinien wrócić do stanu pierwotnego. Oto szkielet kodu do uzupełnienia:

```
#include <atomic>
#include <chrono>
#include <thread>
#include <iostream>
#include <cstdint>

// Wymuszamy wyrównanie obiektów do 4 bajtów, co gwarantuje nam,
// że dwa najmłodsze bity wskaźnika będą zawsze równe 0.
template <typename T>
struct alignas(4) ExchangedNode {
    T payload;
    explicit ExchangedNode(T val) : payload(val) {}
};

template <typename T>
class LockFreeExchanger {
private:
    // Stany maszyny
    static constexpr uintptr_t EMPTY = 0;
    static constexpr uintptr_t WAITING = 1;
    static constexpr uintptr_t BUSY = 2;
    static constexpr uintptr_t STATE_MASK = 3;

    // Atomowa zmienna trzymająca jednocześnie wskaźnik i stan
    std::atomic<uintptr_t> slot{EMPTY};

    // --- FUNKCJE POMOCNICZE (Tagged Pointers) ---
    static uintptr_t pack(ExchangedNode<T>* ptr, uintptr_t state) {
        return reinterpret_cast<uintptr_t>(ptr) | state;
    }
    static ExchangedNode<T>* extract_ptr(uintptr_t val) {
        return reinterpret_cast<ExchangedNode<T>*>(val & ~STATE_MASK);
    }
    static uintptr_t extract_state(uintptr_t val) {
        return val & STATE_MASK;
    }

public:
    // Próbuje dokonać wymiany.
    // Zwraca wskaźnik na węzeł od partnera (sukces) lub nullptr (timeout).
    ExchangedNode<T>* exchange(ExchangedNode<T>* my_node,
std::chrono::microseconds timeout) {
        auto start_time = std::chrono::steady_clock::now();

        while (true) {
            uintptr_t current = slot.load();
            uintptr_t state = extract_state(current);
            ExchangedNode<T>* partner_node = extract_ptr(current);

            // --- MIEJSCE NA TWÓJ KOD ---
```

```
        }
    }
};
```

Wskazówka: The Art of Multiprocessor Programming 2e, r. 11.4.1